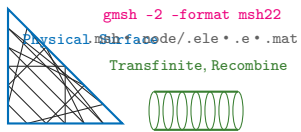
# CREATING MESHES

gmsh by script and by mouse, Triangle and TetGen from `.poly` files, CAD to Coreform Cubit to ExodusII, libMesh's own generators, and the element-size rules reconciled




Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part IV • Chapter 18

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

# LEARNING GOALS

By the end of this chapter you should be able to:

1. say what a mesh must have before IBFE will take it: the right dimension, an element type libMesh knows, elements that go round the right way, a size that fits the fluid grid, and a file format with an extension libMesh recognises;
2. make a 2D mesh with gmsh three ways, by script, by mouse and by command line, and export it in a version libMesh reads;
3. make a 3D mesh with gmsh from a `.geo` file (extrusion, revolution, a transfinite hex tube), and finish Nick Battista's 2014 tube tutorial, which stopped at a surface;
4. write a `.poly` file and mesh it with Triangle (2D) or TetGen (3D), including holes, and read the result;
5. take a CAD model through Coreform Cubit (once Trelis and Bolt) to an ExodusII file whose blocks become libMesh subdomains;
6. build a mesh inside the program with libMesh's generators, as IBAMR's own examples do;
7. choose an element size: the five rules the course taught over ten years, what IBAMR actually does with the mesh, and the rule that follows;
8. recognise the errors a bad mesh produces (`Jacobian ... must be positive`, `mesh_dimension() == spatial_dimension()`, `Unknown data format`, `Unrecognized file extension`) and fix the file.

# WHERE THIS CHAPTER COMES FROM

- Alex Hoover, “2-IBFE Tutorial: Creating your mesh” (2014–2017, 11 slides): element types, the four parts of a `.poly` file, TetGen and `TetGenIO`. Two of its slides, *Holes* and *Generating your elements*, were never written (slides 31 and 29 write them), and its element-type list had names that do not exist (slide 5).
- “2.1 - IBFE Tutorial: Make your own mesh and move it!” (Feb 2017; MATH 769 2018; lab versions 2020 and 2024-08-01, 16 slides): the gmsh GUI homework: a triangle with legs $0.5$, meshed at the fine grid spacing, swapped into the plate example with `MFAC = 1` and `IB_POINT_DENSITY = 1`. Example IBFE/05 is this homework, done.
- Julian Wag, “Using gmsh to make 2D meshes from scratch” (2016, 13 slides with GUI screenshots): points, lines, plane surface, Mesh 2D, the global mesh size factor, Save as `.msh`. Five of its screenshots are reproduced here with credit.
- Nick Battista, “How To Make Simple Geometries in Gmsh” (24 September 2014, 11 pages): a tube from parameters, points at the “poles”, circle arcs, line segments, ruled and plane surfaces; it ends at a surface mesh. The PDF is in every copy of the course folders; its 46 screenshots are not reproduced (slides 21 and 22 transcribe the steps).
- The meshing read-mes (2015–2017): the gmsh links and the sizing rule (“average size should equal Cartesian mesh width”), the Trelis read-me (a shared floating licence; csimsoft tutorials), the 2016 Trelis notes (pave then sweep, merge, wall thickness, 1:1), and the vendor’s *Getting Started with Trelis* PDF (2014, linked, not reproduced). The meshing read-me’s forwarded IT e-mail and licence host are not reproduced.
- The 2020 “Plans for June” deck: the tool list (gmsh, Trelis, Bolt, Meshmixer); the CAD route of the two-coral example (chapter 21).
- New here: every command of this chapter was run with gmsh 4.12.1, Triangle and TetGen (slide 41); the MSH 2.2 / 4.1 / binary question; the two rules a gmsh file must obey; what gmsh physical groups and Exodus blocks become in libMesh; the generators of IBAMR’s own examples; the element-size rules reconciled from the upstream inputs.

## WHAT A MESH MUST HAVE BEFORE IBFE WILL TAKE IT

| property | what IBFE needs |
|---|---|
| dimension | the mesh dimension equals the program's: triangles or quadrilaterals for `main2d`, tetrahedra or hexahedra for `main3d`. A file that holds only lines (or only surface triangles in 3D) fails with `Failed assertion: mesh->mesh_dimension() == mesh->spatial_dimension()`. |
| element type | one of libMesh's (slide 5); the type is in the file (gmsh, Exodus, TetGen) or given when the mesh is built (`ELEM_TYPE`). |
| orientation | every element's nodes must go round so that its Jacobian is positive; IBAMR checks and stops otherwise (slide 17). |
| size and position | in the simulation's units, already placed where it belongs in the box (or moved by an initial coordinate mapping, chapter 22); an element size matched to the fine grid spacing (slide 38). |
| one file, one extension | the extension picks the reader (chapter 17, slide 32): `.msh` gmsh ASCII, `.e` or `.exo` ExodusII, `.node` and `.ele` TetGen, `.mat` MATLAB, `.inp` Abaqus, `.vtu`, `.unv`, `.ucd`, `.off`, `.mesh`, `.xda` or `.xdr`, `.cpa` or `.cpr`. Nothing else is recognised. |
| groups (optional) | gmsh physical groups and Exodus blocks become libMesh subdomain ids and names; lower-dimensional groups become side sets; chapter 21 uses them to give parts of one mesh different materials. |

> "In keeping with the practice of using `.vertex`, `.spring` and `.beam` files for complicated geometries, this tutorial is focused on the TetGen, Triangle and MATLAB tools" (2014). Ten years on, gmsh covers most of it, and Coreform Cubit the CAD end.

# ELEMENT TYPES: LIBMESH'S NAMES, GMSH'S NUMBERS, AND THE 2014 SLIDE'S SLIPS

| dimension | libMesh type | nodes | gmsh type | notes |
|---|---|---|---|---|
| 2D | `TRI3` | 3 | 2 | linear triangle; the course's meshes (IBFE/01, 05, 06, 08, 12) |
| | `TRI6` | 6 | 9 | quadratic triangle; upstream ex4, ex5 |
| | `QUAD4` | 4 | 3 | bilinear quadrilateral |
| | `QUAD8`, `QUAD9` | 8, 9 | 16, 10 | quadratic quadrilaterals; `QUAD9` in upstream ex0–ex3, ex9 |
| 3D | `TET4` | 4 | 4 | linear tetrahedron; gmsh's default in 3D, TetGen's output, IBFE/10 |
| | `TET10` | 10 | 11 | quadratic tetrahedron |
| | `HEX8` | 8 | 5 | trilinear hexahedron; upstream ex2 and ex4 in 3D |
| | `HEX20`, `HEX27` | 20, 27 | 17, 12 | quadratic hexahedra; `HEX27` in upstream ex5 and ex9 in 3D |
| | `PRISM6`, `PRISM15`, `PRISM18` | 6, 15, 18 | 6, 18, 13 | wedges |
| | `PYRAMID5` | 5 | 7 | pyramids |

# ELEMENT TYPES (2): WHAT THE TABLE SAYS, AND THE 2014 SLIDE'S SLIPS

- The gmsh numbers are the ones libMesh's `GmshIO` maps (`gmsh_io.C`, the `ElementDefinition` table); a gmsh type outside this list stops the read.
- The 2014 slide listed "`TET4`, `TET8`" and "Hexagonal (`HEX6`, `HEX8`)". There is no `TET8` (it is `TET10`) and no `HEX6` (a six-node element is a prism); "hexagonal" should be *hexahedral*.
- "Piecewise linear or piecewise cubic" (2014): the choice is linear (`TRI3`, `QUAD4`, `TET4`, `HEX8`) or quadratic (the rest); libMesh has no cubic Lagrange elements in this family.
- A `MATLAB .mat` file is triangles only; TetGen writes `TET4` only; Triangle writes `TRI3` (or 6-node triangles with `-o2`, which libMesh's TetGen reader does not take); gmsh and Cubit write any of these.

# WHAT IBAMR'S OWN EXAMPLES USE

Every upstream IBFE example builds its mesh in the program (slide 10) with an element size `ds = MFAC*DX`; these are their input files in 0.19.0:

| example | `ELEM_TYPE` | `MFAC` | `IB_POINT_DENSITY` | `PK1_DEV` / `DIL_QUAD_ORDER` | what it is |
|---|---|---|---|---|---|
| ex0, ex1 (2D) | `QUAD9` | 4.0 | 2.0 | – | a rubber band (thin ring), stretched band |
| ex2 (2D / 3D) | `QUAD9` / `HEX8` | 2.0 | 2.0 | FIFTH/THIRD, THIRD/FIRST | an elastic block in a channel |
| ex3 (2D) | `QUAD9` | 2.0 | 2.0 | FIFTH/THIRD | a block with a different stress |
| ex4 (2D / 3D) | `TRI6` / `HEX8` | 2.0 / 1.0 | 3.0 | FIFTH/THIRD, THIRD/FIRST | a compressible disc / sphere |
| ex5 (2D / 3D) | `TRI6` / `HEX27` | 8.0 | 4.0 | SEVENTH/FIFTH | a disc / sphere in a stream |
| ex6 (2D) | (input) | 1.0 | 1.0 | – | a disc and a beam (two parts) |
| ex8 (2D) | (input) | – | 2.0 | FIFTH/THIRD | three parts |
| ex9 (2D / 3D) | `QUAD9` / `HEX27` | 4.0 / 8.0 | 4.0 | SEVENTH/FIFTH | a thin shell |
| ex10, ex11 (2D) | `TRI3` | 2.0 / 8.0 | 4.0 | – | direct-forcing rigid bodies |

- `MFAC` runs from $1$ to $8$ and `IB_POINT_DENSITY` from $1$ to $4$: elements up to eight grid cells wide work when the quadrature puts enough points in each cell. That is the key to slide 38.
- Quadratic elements (`QUAD9`, `TRI6`, `HEX27`) carry the larger `MFAC`; linear ones (`TRI3`, `HEX8`) sit at $1$–$2$.
- The course's examples read their meshes from files, so their `MFAC` lines do nothing (chapter 17); their element sizes were chosen when the files were made: $2\,\mathrm{DX}$ for IBFE/01, $1\,\mathrm{DX}$ for IBFE/05, about $3\,\mathrm{DX}$ for IBFE/10's plate.

# FIVE ROUTES TO A MESH

| route | tools | output | used by |
|---|---|---|---|
| A. build it in the program | libMesh `MeshTools::Generation` (`build_square`, `build_cube`, `build_sphere`) | none: the mesh exists in memory | every upstream IBFE example (slide 10) |
| B. gmsh | a `.geo` script or the GUI; `gmsh -2`/`-3` | `.msh` (ASCII 2.2 or 4.1) | IBFE/05 (2D), IBFE/10 (3D); chapters 21, 22 |
| C. Triangle / TetGen | a `.poly` file; `triangle -pq`, `tetgen -pq` | `.node` + `.ele` (+ `.face`, `.edge`) | the 2014 tutorial; the course's `Beam_IBFE_Mesh2D.m` |
| D. CAD to a mesher | Fusion 360, SolidWorks, Blender, SpaceClaim → STEP or STL → Coreform Cubit (or gmsh) | `.e` ExodusII (or `.msh`) | IBFE/08 (Trelis); the two corals (chapter 21); the carotid artery (coming) |
| E. MATLAB PDE Toolbox | `pdepoly`, `initmesh`, a `fprintf` loop | `.mat` (triangles) | IBFE/01, 06, 12 (chapter 17, slide 39) |

# FIVE ROUTES TO A MESH (2): WHICH TO LEARN

- Route A needs no file and no other program; it makes rectangles, boxes, discs and spheres, and anything you can deform them into with a loop over the nodes (ex6 shifts a disc by $0.2$ that way).
- Routes B and C are free and scriptable; B is the one to learn.
- Route D is for shapes that start as CAD: scans, drawings, bodies with fillets and holes. Cubit also makes hexahedral meshes, which gmsh does only for simple shapes.
- Whatever the route, the file goes through `mesh.read()` and the checks of slide 39.

# ROUTE A: LIBMESH'S GENERATORS, AS IBAMR'S EXAMPLES USE THEM

Upstream `IBFE/explicit/ex2` (an elastic block $0.1$ wide and $1$ tall, at $0.95 < x < 1.05$):

```
Mesh mesh(init.comm(), NDIM);
const double dx = input_db->getDouble("DX");
const double ds = input_db->getDouble("MFAC") * dx;            // the element size: MFAC fine-grid cells
string elem_type = input_db->getString("ELEM_TYPE");
#if (NDIM == 2)
MeshTools::Generation::build_square(mesh,
                                    static_cast<int>(ceil(0.1 / ds)),   // elements across
                                    static_cast<int>(ceil(1.0 / ds)),   // elements up
                                    0.95, 1.05,                         // xmin, xmax
                                    0.0, 1,                             // ymin, ymax
                                    Utility::string_to_enum<ElemType>(elem_type));
#endif
#if (NDIM == 3)
MeshTools::Generation::build_cube(mesh,
                                  static_cast<int>(ceil(0.1 / ds)), static_cast<int>(ceil(1.0 / ds)), static_cast<int>(ceil(1.0 / ds)
  ),
                                  0.95, 1.05, 0.0, 1, 0.0, 1,
                                  Utility::string_to_enum<ElemType>(elem_type));
#endif
```

- `nx`, `ny` (`nz`) are element counts, so `ceil(length / ds)` turns a size into a count; the element type comes from the input as a string, through `Utility::string_to_enum<ElemType>`.

# ROUTE A (2): THE GENERATORS' SIGNATURES, AND PLACING THE RESULT

The signatures (libMesh 1.7.8, `mesh_generation.h`):

```
void build_square(UnstructuredMesh& mesh, unsigned int nx, unsigned int ny, Real xmin = 0., Real xmax = 1., Real ymin = 0., Real
  ymax = 1., ElemType type = INVALID_ELEM, bool gauss_lobatto_grid = false);
void build_cube(UnstructuredMesh& mesh, unsigned int nx = 0, unsigned int ny = 0, unsigned int nz = 0, Real xmin = 0., Real xmax =
  1., Real ymin = 0., Real ymax = 1., Real zmin = 0., Real zmax = 1., ElemType type = INVALID_ELEM, bool gauss_lobatto_grid = false);

void build_sphere(UnstructuredMesh& mesh, Real rad = 1, unsigned int nr = 2, ElemType type = INVALID_ELEM, unsigned int n_smooth = 2,
   bool flat = true);
```

- `build_sphere` makes a disc in 2D and a ball in 3D with $4 \cdot 2^r$ segments around; ex6 chooses $r$ from `ds`: `num_circum_segments = ceil(2 pi R / ds); r = log2(0.25 * num_circum_segments)`, then shifts every node by $(0.2, 0.2)$ in a loop over `block_mesh.nodes_begin()`. That loop is the generator route's answer to "how do I place it?".
- Boundary ids are set to the side indexing of the master element ("`build_square`: boundary ids are set to be equal to the side indexing on a master quad"), which ex2 then relabels in a loop over the elements' sides.

# ROUTE B: GMSH IN ONE SLIDE

gmsh (https://gmsh.info, open source) builds geometry from points, curves, surfaces and volumes, in a text file (`.geo`) or by mouse, and meshes it. The pieces of a `.geo` file:

```
lc = 0.0156;                                  // a variable: the target element size
Point(1) = {0.5, 0.0, 0.0, lc};               // x, y, z, element size at the point
Line(1) = {3, 2};                             // a straight line between two points
Circle(1) = {2, 1, 3};                        // an arc: start point, centre, end point (less than a half turn)
Curve Loop(4) = {1, 2, -3};                   // a closed loop of curves, minus = reversed ("Line Loop" in gmsh 2 and 3)
Plane Surface(5) = {4};                       // a flat surface bounded by the loop (holes: more loops)
Surface(14) = {13};                           // a curved surface ("Ruled Surface" in gmsh 2 and 3)
Surface Loop(29) = {28, 26, 16, 14};          // a closed shell of surfaces
Volume(30) = {29};                            // the solid inside it
Extrude {0, 0, 1} { Surface{6}; }             // sweep a surface into a volume (translation)
Extrude {{0,0,1}, {0,0,0}, Pi/2} { Surface{5}; }   // revolve a surface about an axis
Physical Surface("triangle", 1) = {5};        // the group the mesh file will keep (2D)
Physical Volume("tube") = {179, 181};         // the same in 3D
Transfinite Line {1, 2} = 24; Transfinite Surface "*"; Recombine Surface "*"; Transfinite Volume "*";   // a structured hex mesh
Mesh.Algorithm = 5;                           // 2D algorithm: 5 Delaunay, 6 Frontal-Delaunay (gmsh 4's default)
```

- Every entity has a number; the numbers are shared across curve types (“even though it says `Line(12)`, we have only defined 4 lines so far”: Battista), so take the next free one or let the GUI do it.
- A `.geo` file is reloaded with **Reload** after every edit; the GUI writes the same commands into the file when you click, so you can mix the two (Battista’s method, slide 21).
- The file’s element size is `lc` at the points, scaled by **Global mesh size factor** (`Mesh.MeshSizeFactor`, slide 19).

# THE TRIANGLE OF IBFE/05: TRIANGLE.GEO

```
// triangle.geo: gmsh script for the triangle of example IBFE/05.
// Make a mesh file that libMesh can read (ASCII text, MSH format 2.2):
//   gmsh -2 -format msh22 triangle.geo -o triangle_gmsh.msh
// libMesh reads ASCII MSH files (format 2.2 or 4.1) but not binary ones.

// Target element size: about one finest grid cell, DX = L/NFINEST = 2/128.
lc = 0.0156;
// 2D meshing algorithm: 5 is Delaunay. With it, gmsh 4.12 makes the same
// numbers of nodes and triangles as the course mesh (759 and 1404). Without
// this line gmsh 4 uses Frontal-Delaunay (6): 687 nodes and 1260 triangles.
Mesh.Algorithm = 5;

Point(1) = {0.5, 0.0, 0.0, lc};
Point(2) = {0.0, 0.0, 0.0, lc};
Point(3) = {0.0, 0.5, 0.0, lc};

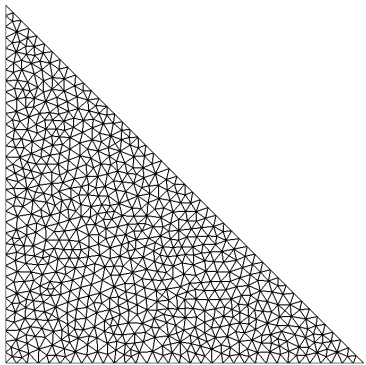
Line(1) = {3, 2};          // left side, on x = 0
Line(2) = {2, 1};          // bottom side, on y = 0
Line(3) = {3, 1};          // the long side

Curve Loop(4) = {1, 2, -3};  // "Line Loop" in gmsh 3 and older
Plane Surface(5) = {4};

// Once a file has any physical group, gmsh saves only the elements that
// belong to a physical group. A 2D mesh therefore needs a Physical Surface.
Physical Surface("triangle", 1) = {5};
```

Twenty lines: a size, an algorithm, three points, three lines, a loop, a surface, a group. The comments are the file's own; they record what was found when the course's `triangle.msh` was reproduced (next slide).

# THE TRIANGLE OF IBFE/05 (2): WHAT THE SCRIPT MAKES



The 2017 homework's triangle, "corners $(0.5, 0)$, $(0, 0)$ and $(0, 0.5)$" ("Remember you want to select these so they will fit in your domain": $L = 2$, from $-1$ to $1$), meshed at the fine grid spacing: $759$ nodes, $1404$ triangles, the same counts as the course's `triangle.msh`. "We want the ratio of the finite element mesh to the Cartesian mesh to be 1:1. We will therefore pick a meshwidth of $L/\mathrm{NFINEST} = 2.0/128 = 0.0156$."

# RUNNING GMSH FROM THE COMMAND LINE, AND WHICH MSH VERSION TO WRITE

```
gmsh -2 -format msh22 triangle.geo -o triangle.msh      # 2D mesh, MSH 2.2 ASCII
gmsh -3 -format msh22 cube.geo -o cube.msh              # 3D mesh
gmsh -2 triangle.geo -o triangle41.msh                  # gmsh 4's default: MSH 4.1 ASCII (libMesh
  reads it too)
gmsh -2 -format msh22 -bin triangle.geo -o bad.msh      # binary: libMesh cannot read it
gmsh tube.geo -0 -format msh22                          # run a script that meshes and saves itself
  ("Mesh 3; Save ...")
gmsh plate2.msh -save -format vtk -o plate2.vtk         # convert a mesh (here for VisIt)
```

| format | libMesh 1.7.8 |
|---|---|
| MSH 2.2 ASCII | read and written; the safe choice; what every example here ships (`$MeshFormat` line: `2.2 0 8`) |
| MSH 4.0, 4.1 ASCII | read since libMesh 1.5 (“Mesh version 4.0 is a near complete rewrite”); gmsh 4 writes 4.1 unless told otherwise (`4.1 0 8`) |
| MSH 1.0 | read (“`version <= 1.0`” branch) |
| any binary MSH | not read: `Error: Unknown data format for mesh in Gmsh reader.` (`2.2 1 8`: the middle 1 means binary) |

# MSH VERSIONS (2): THE GUI'S EXPORT DIALOG, AND "VERSION ???"

- In the GUI: **File** → **Export**, choose *Mesh - Gmsh MSH (*.msh)*, and in the dialog pick *Version 2 ASCII* (the 2017 slide's "choose the .msh file type in the drop down list" predates the version dialog; Wag's 2016 slide used *Save as*).
- The 2020 and 2024 decks say "Download gmsh, version ???; we previously used 2.9.0." Any gmsh 4 works; the syntax changes are the names in slide 12, which gmsh 4.12 still accepts.

# THE TWO RULES A GMSH FILE MUST OBEY, AND THE ERRORS WHEN IT DOES NOT

1. Go round the surface counter-clockwise. gmsh numbers each triangle's corners in the direction of the curve loop. A clockwise loop (`{3, -2, -1}` for the triangle) makes every triangle clockwise, and IBAMR stops at start-up:

```
IBAMR requires that the Jacobian of each element must be positive, but the present element has at least one point at which the
   Jacobian is either negative or zero.
... (the element's corners) ...
To get around this error, you will need to edit the problematic mesh either by hand or with a CAD program.
```

   Fixes: reverse the loop; `ReverseMesh Surface{5};` in the `.geo`; or `python3 fix_orientation.py old.msh new.msh` (IBFE/05 ships it: it swaps the last two nodes of every clockwise triangle and changes nothing else). The course's `triangle.msh` had all $1404$ triangles clockwise; IBAMR has checked since 0.11.0 (2022).

2. Put the surface (or volume) in a physical group. Once a file has any physical group, gmsh saves only elements in physical groups. A file with only a `Physical Curve` holds the $112$ boundary lines and no triangles, and the mesh is one-dimensional:

```
Failed assertion: mesh->mesh_dimension() == mesh->spatial_dimension()
```

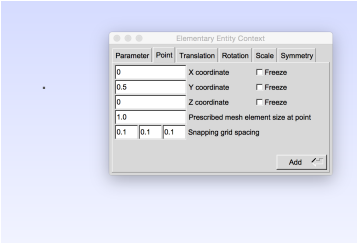
   With no physical group at all, gmsh saves everything, including the points and lines (slide 23); libMesh turns the lower-dimensional elements into boundary information, so this also works, but the file is bigger.

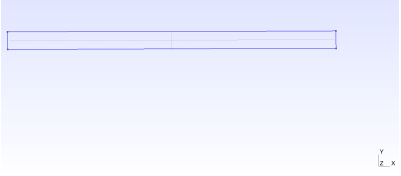
All three cases of this slide were run with IBAMR 0.16.0 and 0.19.0 (IBFE/05's README) and give these messages.

# THE GUI ROUTE (1): POINTS, LINES, A PLANE SURFACE

Julian Wag's 2016 steps, kept by the 2017–2024 homework decks:

1. **Modules** → **Geometry** → **Elementary entities** → **Add** → **Point**: type the coordinates, or place the point with the mouse and press **e**; **q** ends the mode. "If using the mouse, change the snap value for more or less precision." Set *Prescribed mesh element size at point* to the element size ($0.0156$ in the homework).
2. **Add** → **Straight line** (or Spline, Circle arc): click the two end points of each line.
3. **Add** → **Plane surface**: click the closed border and press **e**; "the plane is indicated by dashed lines in the middle of the border lines."

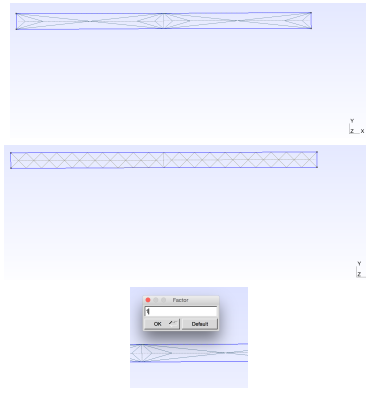


Screenshots: Julian Wag, "Using gmsh to make 2D meshes from scratch" (2016), gmsh 2.x on macOS.

# THE GUI ROUTE (2): MESH, REFINE, SAVE

4. **Modules** → **Mesh** → **2D**. “For some strange reason, you should click 1D and then click 2D to make it show up” (2017); both decks say the first mesh “might not look good, e.g. the tris are not even, the mesh is not refined enough.”
5. Right- and left-click together in the canvas (or **Tools** → **Options** → **Mesh** → **General**): **Global mesh size factor**, type a factor (default $1$), OK; then **1D**, **2D** again to refresh.
6. Wag’s example: a rectangle $0.1 \times 2$ with the factor $0.1$ “allowed even tris, with 1 on the short length and 20 on the long length”: $80$ triangles, “four times as many triangles as tri edges in the long edge.”
7. **File** → **Export** (2017: “Save as”), `.msh`, Version 2 ASCII. “Put this `.msh` file in the folder of your simulation and modify the `main.c` file to read in the mesh”: today, `MESH_FILENAME = "triangle.msh"` in `input2d`.




Screenshots: Julian Wag (2016). Top: the first mesh; middle: after the size factor $0.1$.

# THE HOMEWORK'S LAST TWO SLIDES: WHAT TO CHANGE IN THE PROGRAM AND THE INPUT

- "In `main.C`, search for `IBFE_Mesh2D_128.mat`, replace with `triangle.msh`" (2017–2024). In the shipped examples the name is the input key `MESH_FILENAME`; nothing is recompiled.
- "Changes to `input2d`: `MFAC = 1.0`, `IB_POINT_DENSITY = 1.0`. Did this to update to 1:1 ratio for boundary mesh size to Cartesian mesh size. You may also need to change `DT` to something smaller." The `MFAC` line does nothing to a read-in mesh (chapter 17); the point density does, and IBFE/05 keeps $1.0$ because its elements are one grid cell across (slide 38). Its `DT` is $5 \times 10^{-4}$, half of IBFE/01's: the smaller elements and the sharper corners of the triangle move faster fluid near them.
- "Run simulation: here is mine with both grids!" (the slide showed the triangle on the AMR patches). IBFE/05's README has the numbers: the centre rises from $y = 0.168$ to $0.668$ at $t = 1$ (the top corner reaches $0.997$, just under the top of the box), returns, is released at $t = 3$ and sinks to $-0.740$ by $t = 4$, turning as it goes; every node stays within $0.011$ of its target while tethered.
- The 2017 copy of the homework was a zip, `Example_IBFE_TetherForceFullTriangle2D_2016-11.zip`, with `Beam_IBFE_Mesh2D.m` and the `.mat` plate still inside, unused: the same mismatch as chapter 17's extras.

> A new mesh is a new structure: check that it fits the box, re-choose the point density and the time step, and expect different numbers. The homework's own warning was "you may also need to change `DT`."

# BATTISTA'S TUBE TUTORIAL (1): PARAMETERS, POLES, ARCS

*How To Make Simple Geometries in Gmsh* (Nick Battista, 24 September 2014) builds a tube by mouse and text together. Its steps, with the `.geo` lines they produce:

1. **File** → **Save As** `geometry.geo` first ("the first obstacle: getting the file to save where you want it"), then **Modules** → **Geometry** → **Reload**.
2. **Elementary entities** → **Parameters**: $L = 2.0$ (length), $r = 0.25$ (radius). Then **Edit file** ("once you get the hang of gmsh, this will be your best friend"; Windows users: set the editor command to `C:/Windows/notepad.exe %s`) and add an element size:

   ```
   ds = DefineNumber[ L/100.0, Name "Parameters/r" ];   // the tutorial's line; the Name should be "Parameters/ds"
   ```

   The name is the label in the GUI's parameter list; reusing `"Parameters/r"` ties `ds` to the slider called $r$, so the tutorial's own figure shows two parameters named $r$. Write `"Parameters/ds"`.
3. **Point**: five points for the first circle, the centre and the four "poles" (N, S, E, W) at $z = +L/2$, each with `ds` as the prescribed mesh element size ("add `ds` to prescribed mesh element size at point"); the other four typed into the file.
4. **Circle arc**: three clicks per arc, "the point where the arc starts, the central point, the point where the arc ends" (an arc is less than a half turn, so a circle needs four); "I am doing everything going counter-clockwise. It's good just to keep a constant convention."
5. The second circle at $z = -L/2$ "completely in the text-file"; then four straight lines between matching poles: `Line(11) = {3, 8}; Line(12) = {4, 9};` and the note on shared numbering (slide 12).

# BATTISTA'S TUBE TUTORIAL (2): SURFACES, AND WHERE IT STOPS

6. **Ruled Surface** (gmsh 4: **Surface**): click `Circle(1)`, `Line(9)`, `Circle(5)`, `Line(10)`, press **e**: one quarter of the curved wall; the other three the same way or in the file. "Note the dotted lines signifying a surface."
7. **Mesh** → **2D**: the wall is meshed with triangles.
8. For a closed tube, **Plane Surface** on the four arcs of each end; **e**.

The tutorial ends there, with a surface mesh and no file saved. What IBFE needs is a *volume* mesh in a file:

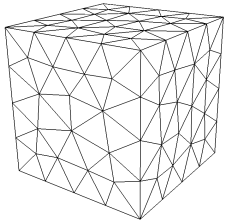
```
Surface Loop(100) = {14, 16, 18, 20, 21, 22};   // the four wall pieces and the two caps
Volume(101) = {100};
Physical Volume("tube") = {101};                 // so that only the tetrahedra are saved
// then:  gmsh -3 -format msh22 geometry.geo -o tube.msh
```

- A hollow tube (a vessel wall) is two concentric circles per end and the same construction twice; the course's `tube.geo` (slide 23) does it with transfinite hexahedra instead, and chapter 24's tube example uses a 2D section.
- The crosswalk's other notes on this document: `Ruled Surface` and `Line Loop` are gmsh-2 names (accepted by gmsh 4.12 with no warning in the runs of slide 41); the ONELAB parameter name is duplicated (above); the tutorial is Battista's and is cited, not copied.

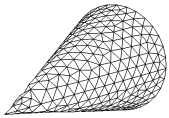
# THE COURSE'S FOUR SAMPLE `.GEO` FILES, RUN TODAY

| file | what it does | `gmsh -3 -format msh22`, gmsh 4.12.1 |
|---|---|---|
| `cube.geo` | a unit square (`lc = 0.3`) extruded by $(0, 0, 1)$ into a volume; `Physical Volume(1) = tmp[1];` takes the volume the extrusion returned | $141$ nodes, $373$ `TET4`; nothing else saved, because of the physical volume |
| `cone.geo` | a triangle in the $xz$-plane revolved four times by $\pi/2$ about the $z$ axis (`Extrude {{0,0,1}, {0,0,0}, Pi/2} { Surface{5}; }` four times, each on the surface the last one produced); no physical group | $648$ nodes, $3845$ elements: $2460$ `TET4`, $1248$ `TRI3`, $131$ lines, $6$ points. libMesh reads the tetrahedra as the mesh and files the rest as boundary information; add `Physical Volume` to keep the file small |
| `sphere.geo` | eight circle arcs on three great circles, eight `Ruled Surfaces`, a surface loop, a volume; `Physical Surface` and `Physical Volume`; `Mesh.Algorithm = 6`; `lc = .55` | $60$ nodes, $114$ `TET4` and $114$ `TRI3` (the surface group): far too coarse for a body in a flow, a demonstration of the construction |
| `tube.geo` | a tube of length $10$ and diameter $1$ built from $34$ points and $73$ curves, `Transfinite` everything, `Recombine Surface "*"`, `Transfinite Volume "*"`, physical groups `inlet`, `outlet`, `walls`, `tube`; ends with `Mesh 3; Save "tube.msh";` | $593{,}800$ nodes, $579{,}488$ `HEX8` and $28{,}112$ `QUAD4`; $80$ MB. The densities `den1 = 15`, `den2 = 20`, `den3 = 200` are the counts across, radially and along; $6/5/24$ give the picture's $2{,}784$ hexahedra |

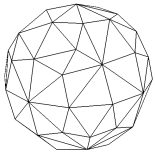
# THE FOUR SAMPLES, MESHED



`cube.geo`: extrusion, $373$ tets



`cone.geo`: four revolutions, $2460$ tets



`sphere.geo`: $114$ tets at `lc = 0.55`



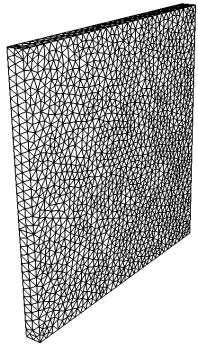
`tube.geo` at $6/5/24$: $2784$ hexes

- Rendered in VisIt from VTK copies (`gmsh x.msh -save -format vtk -o x.vtk`); VisIt does not read `.msh` itself. Chapter 27 does the same with the 3D results.
- The tube is the only hexahedral mesh in the course material; everything from gmsh's free meshers is tetrahedra. Hexahedra need a structured construction (transfinite blocks, as here) or Coreform Cubit (slide 35).
- These four files sit next to Battista's PDF in every copy of the meshing folder, with no read-me of their own; they are the "examples" the Trelis read-me and the meshing read-me refer to. No warnings and no errors from gmsh 4.12.1 for any of them.

# A 3D GMSH MESH IBFE ACTUALLY RUNS: THE PLATE OF IBFE/10

```
$MeshFormat
2.2 0 8
$EndMeshFormat
$PhysicalNames
1
3 1 "plate2"
$EndPhysicalNames
$Nodes
3386
1 0 1 1
2 0 -1 1
3 0 -1 -1
...
```

- `plate2.msh` (2017): a plate $0.125$ thick and $2 \times 2$, $3{,}386$ nodes, $11{,}481$ `TET4`, one physical volume named `plate2`; element size about three fine cells ($\Delta x = 1/16$ in the tank). Its `.geo` is lost; the 2020 flow-tank deck (chapter 22) shows the GUI steps that made it.
- The physical name becomes the libMesh subdomain name (`mesh.subdomain_name(id) = phys_name`, `gmsh_io.C`); `mesh.get_id_by_name("plate2")` returns the id.



# GMSH PHYSICAL GROUPS AND WHAT LIBMESH MAKES OF THEM

| in the `.geo` | in libMesh after `mesh.read()` (`gmsh_io.C`, libMesh 1.7.8) |
|---|---|
| `Physical Surface("triangle", 1)` in a 2D mesh, `Physical Volume("plate2")` in 3D | the elements get `subdomain_id = 1`; `mesh.subdomain_name(1) = "triangle"`. Several groups: several subdomains, which chapter 21 uses to give parts of one mesh different stresses (`elem->subdomain_id()` inside a stress function). |
| `Physical Curve` in 2D, `Physical Surface` in 3D (a group one dimension below the mesh) | the elements are not kept as elements; each becomes a side set entry on the element it borders, named after the group (`sideset_name`); nodes of a `Physical Point` become a node set. These are the boundary ids a program can test with `boundary_info.has_boundary_id(elem, side, id)`. |
| a lower-dimensional group whose name contains `lower_dimensional_block` | kept as elements of a lower dimension (libMesh's escape hatch for fractures and shells); not for IBFE. |
| no physical group at all | every element of every dimension is saved and read; the highest dimension is the mesh, the rest become boundary information. Works, but `cone.msh` was three times its necessary size. |

> One rule covers it: give the body a `Physical Surface` (2D) or `Physical Volume` (3D), and give any boundary you want to name a physical group one dimension lower. The IBFE examples here need only the first.

# ROUTE C: THE `.poly` FILE

Triangle (Jonathan Shewchuk, 2D) and TetGen (Hang Si, 3D) mesh a *piecewise linear complex* written in a `.poly` file: “The `.poly` alone is not the mesh but rather a file that you input to the mesh generator” (2014). Its four parts:

| part | 2D (Triangle) | 3D (TetGen) |
|---|---|---|
| 1. nodes | `<# points> <2> <# attributes> <marker 0/1>`, then `<#> <x> <y> [attr] [marker]` | `<# points> <3> <# attributes> <marker 0/1>`, then `<#> <x> <y> <z> [attr] [marker]` |
| 2. segments / facets | `<# segments> <marker 0/1>`, then `<#> <node> <node> [marker]` | `<# facets> <marker 0/1>`, then for each facet `<# polygons> [<# holes>] [marker]`, each polygon `<# corners> <node> ...`, then the facet’s hole points |
| 3. holes | `<# holes>`, then `<#> <x> <y>`: one point inside each cavity | `<# holes>`, then `<#> <x> <y> <z>` |
| 4. regions (optional) | `<# regions>`, then `<#> <x> <y> <attribute> <max area>` | `<# regions>`, then `<#> <x> <y> <z> <attribute> <max volume>` |

- “The first three parts are mandatory, but the fourth part is optional” (2014); a file with no holes still needs the line `0`.
- Node numbers start at `1` (or `0`: the meshers follow the first node’s number; libMesh’s reader handles both).
- A marker on a segment or facet is a boundary id; a region attribute becomes a subdomain id (libMesh’s `TetGenIO` reads it as the element’s `subdomain_id` when the `.ele` file carries attributes).

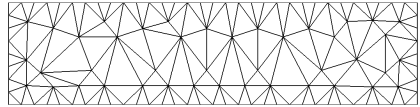
# TRIANGLE ON THE COURSE'S `.poly`

`IBFE_Mesh2D_128.poly`, written by `Beam_IBFE_Mesh2D.m` (chapter 17, slide 40): a $1.0 \times 0.25$ rectangle with $84$ nodes along its outline, no holes:

```
84 2 0 1
1 -5.0000000000000000e-01 1.2500000000000000e-01 1
2 -4.7500000000000000e-01 1.2500000000000000e-01 1
...
84 2 0 1
1 1 2 1
...
0
```

```
triangle -pq IBFE_Mesh2D_128.poly
# -p: read a .poly; -q: quality mesh (no angle
  below 20 degrees)
# -a0.001 would limit the triangle area; -o2
  makes 6-node triangles
```

Output: `IBFE_Mesh2D_128.1.node` ($122$ nodes), `.1.ele` ($162$ triangles), `.1.poly`. The `.1` is Triangle's iteration number.



The $162$ triangles Triangle makes with `-pq` and no area limit: one row across the width, since the outline's spacing ($0.025$) sets the size at the edges and nothing refines the interior.

- libMesh's `TetGenIO` is three-dimensional (it sets `mesh_dimension(3)`), so a Triangle result does not go straight into `main2d`. Convert it (slide 32) or make the 2D mesh with gmsh.
- libMesh can also call Triangle and TetGen itself (`TriangleInterface`, `TetGenMeshInterface`) when it was built with them; autoibamr's libMesh is built without, so the command line is the way.

# TETGEN ON A UNIT CUBE, AND THE “GENERATING YOUR ELEMENTS” SLIDE THAT WAS NEVER WRITTEN

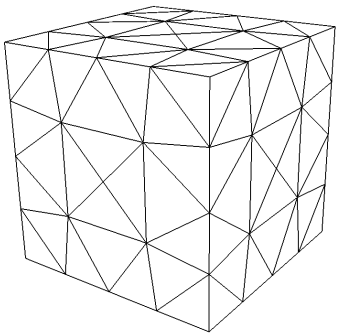
`cube.poly`: eight corners, six square facets, no holes, no regions:

```
# a unit cube: 8 nodes, 6 facets, no holes
8 3 0 0
1 0 0 0
2 1 0 0
3 1 1 0
4 0 1 0
5 0 0 1
6 1 0 1
7 1 1 1
8 0 1 1
6 0
1
4 1 2 3 4
1
4 5 6 7 8
1
4 1 2 6 5
1
4 2 3 7 6
1
4 3 4 8 7
1
4 4 1 5 8
0
0
```

```
tetgen -pq1.4a0.02 cube.poly
# -p: read a .poly; -q1.4: quality (radius-edge
   ratio <= 1.4)
# -a0.02: no tetrahedron larger than 0.02; -k:
  also write a .vtk
```

Output: `cube.1.node` (75 nodes), `cube.1.ele` (153 tetrahedra), `cube.1.face`, `cube.1.edge`.

# TETGEN ON A UNIT CUBE (2): THE RESULT, AND HOW A FACET IS WRITTEN



Each facet line is `<# polygons> [holes] [marker]` then `<# corners> <nodes>`; a facet with a hole lists two polygons (the outer and the inner boundary) and then the hole point inside the facet. The `-a` value is what sets the element size; without it TetGen only fills the hull with the fewest tetrahedra it can.

# HOLES AND REGIONS: THE OTHER SLIDE THAT WAS NEVER WRITTEN

A 2D plate with a round hole, for Triangle (`plate_hole.poly`): the outline, a 12-gon inside it, and one hole point:

```
16 2 0 1          # 4 corners + 12 points on the hole
1 -0.5 -0.125 1
2  0.5 -0.125 1
3  0.5  0.125 1
4 -0.5  0.125 1
5  0.06 0.0 2     # the 12 hole points, radius 0.06, marker 2
...
16 1              # 16 segments: 4 outer + 12 inner
1 1 2 1
...
1                 # one hole
1 0.0 0.0         # any point inside the hole
```

```
triangle -pqa0.0005 plate_hole.poly
```

Triangle removes every triangle it can reach from the hole point without crossing a segment: that is what “a list of hole points” means.

- In 3D the same: a cavity is a closed set of facets and one point inside it, listed in part 3. A *facet* hole (a window in a flat face) is listed inside the facet, after its polygons.
- Regions (part 4) do the opposite of holes: a point inside a closed region gives all elements there an attribute (a subdomain id for libMesh) and a size limit. Two regions in one file: a stiff core and a soft rind, or a stem and a leaf (chapter 22’s two-stiffness plate does this with a position test instead).
- IBFE does not need holes for fluid passages: the fluid is everywhere already, and a hole in the solid is simply fluid. Use a hole for a geometric hole in the body.
- The 2014 slides “Holes” and “Generating your elements” were headings with nothing under them; this slide and the last are what they would have said.

# READING TETGEN (AND TRIANGLE) OUTPUT

The 2014 code, which still compiles once `Mesh mesh(NDIM)` becomes the communicator form:

```
#include <libmesh/tetgen_io.h>
...
Mesh mesh(init.comm(), NDIM);
TetGenIO TETGEN(mesh);
// the course wrote "Jellyfish_IBFE_Mesh3D.ele";
// that file is in no archive
TETGEN.read("example_IBFE_Mesh3D.ele");
mesh.prepare_for_use();
```

- `TetGenIO::read(name)` takes either name of the pair and derives the other (`tetgen_io.C`: "Check name for `*.node` or `*.ele` extension"); both files must be there, or `Error while opening either ... or ....` `mesh.read("cube.1.ele")` does the same through `NameBasedIO`, so the explicit `TetGenIO` is optional.
- It is 3D only ("tetgen only works in 3D"): Triangle's 2D `.node`/`.ele` pair must become something else for `main2d`. Twenty lines of Python turn it into MSH 2.2 (the format of slide 25: a `$Nodes` block with `<#> <x> <y> 0`, an `$Elements` block with `<#> 2 2 1 1 <n1> <n2> <n3>`), or into legacy VTK for a look in VisIt (`wsl/final/ch18_render.sh` does the latter for the plate above). Or skip Triangle: gmsh meshes the same outline from a `.geo` with the hole as a second curve loop.
- A TetGen `.ele` file with region attributes (`tetgen -pA`) gives each tetrahedron a `subdomain_id`; without `-A` every element is subdomain $0$.
- Node and element numbering start at $1$ in TetGen's output unless the input started at $0$; libMesh's reader renumbers either way.

# ROUTE D: FROM A CAD MODEL TO A MESH

| stage | tools and notes |
|---|---|
| make the solid | Fusion 360 (the two corals of chapter 21), SolidWorks, Blender (the hollow carotid of the coming example), SpaceClaim; or a scan (micro-CT, laser) that gives a surface |
| export | STEP or IGES for a true solid; STL for a triangulated surface (what scans and Blender give). The 2020 tool list also had Meshmixer for cleaning STL surfaces; Autodesk discontinued it in 2021, and Blender's remesh tools or MeshLab do the job now. |
| mesh the solid | Coreform Cubit (formerly Trelis, before that Bolt and Cubit from csimsoft/Sandia): imports STEP and STL, makes tetrahedra or, with the pave-and-sweep workflow, hexahedra; exports ExodusII (`.e`) with named blocks and side sets. gmsh: imports STEP (`Merge "part.step";`) and STL (`Merge "part.stl";` then `Surface Loop`, `Volume` on the classified surface); tetrahedra only. |
| read | `mesh.read("part.e")`: Exodus blocks become subdomains (next slide); `mesh.read("part.msh")` as above. |
| check | slide 39: dimension, size, orientation, position, element count. |

# ROUTE D (2): TRELIS, BOLT, COREFORM CUBIT, AND WHAT IS ONLY LISTED

- The 2016 Trelis read-me: “Boyce, Jeremy and I have a single floating license to Trelis”; “Bolt, an improved Trelis” (the licence was later confirmed, per the 2017 Read Me: “We now have a license for Bolt”). Coreform bought Trelis in 2020 and sells it as Cubit; the csimsoft tutorial links in the read-mes are dead, https://coreform.com has their successors. The vendor’s *Getting Started with Trelis* PDF (2014, a brick with a hole, 116 screenshots) is third-party material and is only linked.
- The only Coreform Cubit material of Laura’s own is a July 2022 recording (listed, chapter 0.1); chapter 21 reproduces the Fusion 360 $\rightarrow$ Bolt hex-mesh route of the two-coral example from its 2018 slides.

# ROUTE D (3): THE 2016 TRELIS NOTES, AND EXODUSII IN LIBMESH

From the lab's 2016 "Trellis Notes" (private Drive; reported by the crosswalk, not reproduced in full):

- hex-mesh a body by *paving* one face with quadrilaterals and *sweeping* it through the volume;
- *merge* coincident surfaces of adjacent volumes so the mesh is conforming;
- keep walls at least $4\,\Delta x$ thick, so that there are elements across the thickness and the fluid cannot see through;
- FE size : Navier–Stokes grid size $= 1{:}1$.

The first two are Cubit workflow, still current; the last two are element-size rules, reconciled on slide 38.

What libMesh does with an Exodus file (`exodusII_io.C`):

- each *element block* becomes a subdomain: `subdomain_id = block id`, `subdomain_name = block name`; the block's element type sets the libMesh type (`TRI3`, `HEX8`, ...);
- *side sets* and *node sets* become boundary ids and names;
- nodal and elemental variables stored in the file can be read into systems (`ExodusII_IO::copy_nodal_solution`), which is how a mesh can carry a fibre direction (chapter 20).

IBFE/08's `beam.e` ($98$ nodes, $126$ `TRI3`) and `circle.e` ($977$ nodes, $1851$ `TRI3`) came from Trelis: one block each, no sets, two-dimensional.



# ROUTE E, BRIEFLY, AND WHEN TO USE WHICH ROUTE

- MATLAB's PDE Toolbox (chapter 17, slide 39): `pdepoly` for the outline, **Mesh** → **Parameters** for the size, **Export Mesh**, a `fprintf` loop; triangles only, 2D only; the course's plate, the active-tension plate (IBFE/06) and the advection–diffusion plate (IBFE/12) were made this way. Fine if you have MATLAB; gmsh does the same without it.

| you want | route |
|---|---|
| a rectangle, box, disc or ball, any size | A: build it in the program (no file to lose; the size follows `DX` automatically) |
| a polygon, a tube, a shape you can describe by points and arcs | B: a `.geo` script (reproducible; the GUI to look) |
| a shape from a drawing you can trace, with holes | B or C: gmsh with curve loops, or Triangle from a `.poly` |
| a body from a scan or a CAD model | D: Coreform Cubit, or gmsh on the STEP/STL |
| hexahedra | D (Cubit), or B with transfinite blocks (`tube.geo`) |
| several bodies in one program | any route, one file (or one `build_*` call) per part; chapter 21 |

> "In general, the more money spent the easier to use and the more versatile" (2014) was true of Trelis against gmsh 2. gmsh 4 is easy to script and reads STEP; Cubit still wins for hex meshes and CAD clean-up.

# ELEMENT SIZE: FIVE RULES THE COURSE TAUGHT

| source | rule |
|---|---|
| 2014 Hoover, slide 4 | "Old: mesh discretizing edge length is $ds = 2\,dx$ (as opposed to $\frac{1}{2}dx$). New: it seems like $ds = dx$ works better." With `IB_POINT_DENSITY = 2.0` and "it seems like 1.0 is better." |
| 2016 gmsh read-me | "Elements should be close to evenly sized. Average size should equal Cartesian mesh width for fluid solver." |
| 2016 Trelis notes | "FE : NS 1:1"; "wall thickness $4\,dx$." |
| 2017–2024 homework | "We want the ratio of the finite element mesh to the Cartesian mesh to be 1:1": $0.0156$ at $\mathrm{NFINEST} = 128$, with `MFAC = 1.0` and `IB_POINT_DENSITY = 1.0`. |
| 2018 Dogwood README (and IBFE/01's) | "maximum edge size two to four times the fine grid spacing" ($0.0312$ for the plate), with `IB_POINT_DENSITY = 2.0`. |
| IBAMR's own examples | `MFAC` $1$ to $8$ with `IB_POINT_DENSITY` $1$ to $4$ (slide 7). |

They look contradictory ($\frac{1}{2}$, $1$, $2$–$4$, up to $8$ grid cells) because they answer different questions: how fine the *elements* must be for the solid mechanics, and how dense the *quadrature points* must be for the fluid–structure coupling. The next slide separates them.

# ELEMENT SIZE: WHAT IBAMR DOES, AND THE RULE THAT FOLLOWS

**What the mesh is for.** The elements carry the displacement and the stress; their size sets how well the body's shape and strain are resolved. A thin body needs at least two elements across its thickness to bend (the plate has two); a corner needs small elements to stay a corner. This is ordinary finite element judgement and does not involve `DX`.

**What the coupling needs.** Velocity is interpolated to, and force spread from, *quadrature points* of the elements. `IB_point_density` is the number of those points per fine grid cell that IBAMR aims at; with adaptive quadrature (the default) it raises the rule's order on every element until each has about `point_density` points per cell of its width. Too few points per cell and fluid leaks through the body; the number of *elements* per cell does not matter to the fluid.

**The rule.**

1. Make the elements as large as the solid mechanics allows: $\mathrm{ds} \approx \mathrm{DX}$ to $2\,\mathrm{DX}$ for linear elements (`TRI3`, `TET4`, `HEX8`); $2$–$4\,\mathrm{DX}$ for quadratic ones; never larger than half the body's thickness.
2. Set `IB_point_density` to $2$ (the upstream default for `MFAC = 2`) and raise it to $3$–$4$ if the body leaks or its elements are coarser than $2\,\mathrm{DX}$.
3. Refine the Cartesian grid, not the mesh, when the flow needs it; refine the mesh when the body's shape does.

So the course's rules were all right for their cases: $1{:}1$ with density $1$ (the homework), $2\,\mathrm{DX}$ with density $2$ (the plate), $8\,\mathrm{DX}$ with density $4$ (upstream ex5's quadratic elements). The one to avoid is the "$\frac{1}{2}\,dx$" of 2014: four times the elements for nothing.

Cost: the number of quadrature points scales as (body volume) $\times$ $\texttt{point_density}^{\mathrm{NDIM}}/\mathrm{DX}^{\mathrm{NDIM}}$, independent of the element size; the FE solve scales with the number of nodes. Coarse elements with dense quadrature are cheap.

# CHECKING A MESH BEFORE THE FIRST RUN

| check | how |
|---|---|
| it reads | a ten-line libMesh program, or the example itself with `END_TIME = 0`: `mesh.read()` fails fast on a bad extension, a binary file or a missing `.node` |
| dimension | the assertion of slide 17; in gmsh, **Tools** → **Statistics** shows how many elements of each dimension |
| element count and type | `$Elements` in the `.msh`; the first line of the `.ele`; `ncdump -h part.e` for Exodus (`num_elem`, `num_el_blk`, `connect1:elem_type`) |
| orientation | IBAMR's Jacobian check at start-up (slide 17); gmsh: **Tools** → **Options** → **Mesh** → **Visibility** → **Normals** |
| size and position | the node coordinates' range (`$Nodes`) against the box: `x_lo`, `x_up` in `CartesianGeometry`; a body that touches a periodic boundary wraps around |
| element size | gmsh **Statistics** gives the edge-length range; compare with `DX` per slide 38 |
| look at it | gmsh itself; VisIt after `gmsh x.msh -save -format vtk -o x.vtk` (or `mesh.write("x.e")` from libMesh); the result files' `output.ex2` after the first dump |

# CHECKING A MESH (2): A TEN-LINE CHECKER

A ten-line checker, to build like any example (`IBAMR::IBAMR2d` brings libMesh):

```
#include <ibtk/IBTKInit.h>
#include <libmesh/mesh.h>
#include <libmesh/mesh_tools.h>
int main(int argc, char* argv[]) { IBTKInit ibtk_init(argc, argv, MPI_COMM_WORLD);
    libMesh::Mesh mesh(ibtk_init.getLibMeshInit().comm(), NDIM); mesh.read(argv[1]); mesh.prepare_for_use();
    mesh.print_info();                                  // dimension, nodes, elements, subdomains, boundary ids
    const auto bb = libMesh::MeshTools::create_bounding_box(mesh);
    libMesh::out << "bounding box " << bb.min() << " to " << bb.max() << std::endl; return 0; }
```

- `print_info()` lists the dimension, the numbers of nodes and elements, the element types, the subdomains and the boundary ids: everything of the table above except the orientation, which IBAMR tests at start-up.
- Build it with the same six-line `CMakeLists.txt` as the examples (`add_executable(meshcheck meshcheck.cpp)`); run it as `./meshcheck part.msh`.

# WHAT WAS RUN FOR THIS CHAPTER

gmsh 4.12.1, Triangle 1.6 and TetGen 1.5.0 (Ubuntu 24.04 packages) in WSL, `wsl/final/ch18_meshtest.sh`; the files are the course's, except `triangle.geo` (IBFE/05) and `cube.poly` (slide 29):

| input | command | result |
|---|---|---|
| `triangle.geo` | `gmsh -2 -format msh22` | 759 nodes, 1404 `TRI3`; MSH 2.2; identical counts to the course's `triangle.msh` |
| `triangle.geo` | `gmsh -2` (no format) | MSH 4.1 (`4.1 0 8`): libMesh reads it |
| `triangle.geo` | `gmsh -2 -format msh22 -bin` | binary (`2.2 1 8`): libMesh does not |
| `cube.geo` | `gmsh -3 -format msh22` | 141 nodes, 373 `TET4` |
| `cone.geo` | `gmsh -3 -format msh22` | 648 nodes; 2460 `TET4` + 1248 `TRI3` + 131 lines + 6 points (no physical group) |
| `sphere.geo` | `gmsh -3 -format msh22` | 60 nodes; 114 `TET4` + 114 `TRI3` |
| `tube.geo` | `gmsh tube.geo -0 -format msh22` (the script saves itself) | 593,800 nodes; 579,488 `HEX8` + 28,112 `QUAD4`; 80 MB, 1.2 million lines; 6/5/24 densities: 2784 hexahedra |
| `IBFE_Mesh2D_128.poly` | `triangle -pq` | 122 nodes, 162 `TRI3` |
| `cube.poly` | `tetgen -pq1.4a0.02 -k` | 75 nodes, 153 `TET4`, plus a `.vtk` |
| `plate2.msh`, `beam.e`, `circle.e` | `gmsh -save -format vtk`; VisIt | the pictures of slides 25 and 35 |

No warnings or errors from gmsh on any of the course's files, gmsh-2 syntax included. The IBAMR-side errors of slide 17 are from IBFE/05's README (run on 0.16.0 and 0.19.0).

# WHAT CAN GO WRONG

| symptom | cause and fix |
|---|---|
| `Error: Unknown data format for mesh in Gmsh reader.` | a binary `.msh`; export ASCII (Version 2 or 4 ASCII) |
| `ERROR: Unrecognized file extension` + the format list | the file's extension is not in libMesh's list; rename or convert |
| `Failed assertion: mesh->mesh_dimension() == mesh->spatial_dimension()` | the file holds only lower-dimensional elements (a `Physical Curve` but no `Physical Surface`; an STL surface read as a 3D mesh); add the physical group of the body's dimension, or mesh the volume |
| `IBAMR requires that the Jacobian of each element must be positive ...` | clockwise triangles (a clockwise curve loop) or inverted tetrahedra; reverse the loop, `ReverseMesh`, or `fix_orientation.py` |
| `Error while opening either x.node or x.ele` | TetGen output needs both files next to each other |

# WHAT CAN GO WRONG (2)

| symptom | cause and fix |
|---|---|
| the body is in the wrong place, or huge, or tiny | the mesh is in its own units; scale and move it in the mesher, or register an initial coordinate mapping (chapter 22) |
| fluid passes through the body | too few quadrature points per cell: raise `IB_point_density`; or the body is thinner than a grid cell |
| the run is far slower than the point example | a mesh much finer than `DX` (the "$\frac{1}{2}dx$" rule) or a huge file like the $580$k-hex tube; coarsen, raise the point density |
| gmsh makes a different number of elements than the course file | the 2D algorithm changed between versions (Delaunay 5 vs Frontal-Delaunay 6); set `Mesh.Algorithm`; counts within a few per cent are normal |
| the GUI mesh does not update | click **1D** then **2D** again (the 2017 and 2016 decks both say so) |
| `DefineNumber` parameters share a name | the ONELAB name is the slider's label; give each its own |

# WHAT THE OLDER DECKS HAD THAT IS NOW PART X

| 2014–2020 | now |
|---|---|
| gmsh 2.9.0 (“version ???” in 2020); `Line Loop`, `Ruled Surface`; **File** → **Save as** `.msh` with no version question | gmsh 4; `Curve Loop`, `Surface` (the old names still work); **Export** with the MSH version dialog: Version 2 ASCII |
| Trelis and Bolt (csimsoft; a shared floating licence on the lab’s Windows machine; `csimsoft.com` links) | Coreform Cubit (https://coreform.com); the vendor PDF linked, not copied |
| Meshmixer for STL clean-up | discontinued (2021); Blender’s remesh, MeshLab |
| `TET8`, `HEX6`, “Hexagonal” | `TET10`, `PRISM6`, hexahedral (slide 5) |
| `Mesh mesh(NDIM); TetGenIO TETGEN(mesh); TETGEN.read( "Jellyfish_IBFE_Mesh3D.ele");` | `Mesh mesh(init.comm(), NDIM); mesh.read("x.ele");` (the jellyfish file never shipped) |
| “modify the `main.C` file to read in the mesh” | `MESH_FILENAME` in the input file |
| `MFAC = 1.0` “to update to 1:1” for a read-in mesh | no effect; the element size is in the file; `IB_point_density` is the knob |
| the meshing read-me’s forwarded IT e-mail about the licence server | not reproduced (privacy); the licence is history |

# EXERCISES

1. Make the IBFE/01 plate ($1.0 \times 0.0312$) in gmsh with `lc = 0.03`, export MSH 2.2, and run `MOTION = "FULL"` with it. Compare the centre positions with chapter 17's table; then halve `lc` and compare again.
2. Take `triangle.geo`, make the curve loop clockwise, mesh, and run IBFE/05 until it stops. Fix the file three ways (the loop, `ReverseMesh`, `fix_orientation.py`) and check that all three meshes are the same.
3. Write the plate-with-a-hole `.poly` of slide 31, mesh it with `triangle -pqa0.0005`, convert the `.node`/`.ele` pair to MSH 2.2 with a script of your own, and run it in IBFE/01's program. What does the hole do to the FULL motion?
4. Finish Battista's tube: add the surface loop, volume and physical volume to the `.geo` he built, mesh it in 3D, and read it with the checker of slide 39. How many tetrahedra at `ds = L/100`?
5. Run `tube.geo` at the three densities $6/5/24$, $8/10/60$ and the original $15/20/200$; tabulate nodes, hexahedra, file size and the time gmsh takes. Which would you use at $\mathrm{DX} = 1/16$?
6. Rebuild IBFE/10's plate ($0.125 \times 2 \times 2$, about $3\,\mathrm{DX}$ elements) from a `.geo` of your own by extruding a square, and compare the node count with `plate2.msh`'s $3{,}386$.
7. Build the same plate inside the program with `build_cube` and `MFAC = 3`, run both for $0.5$ s, and compare the tether force lines of the two runs.

# RULES OF THUMB

- A `.geo` script beats the mouse: reproducible, diffable, and it is what the GUI writes anyway.
- Counter-clockwise loops; a `Physical Surface` or `Physical Volume`; export MSH 2.2 ASCII.
- Elements as large as the solid allows ($\mathrm{DX}$ to $2\,\mathrm{DX}$ for linear elements, two across any thickness); `IB_point_density` $2$, more if it leaks.
- `MFAC` and `ELEM_TYPE` are for meshes built in the program; a file brings its own size and type.
- Check dimension, counts, bounding box and orientation before the first run; IBAMR's own checks catch the last two.
- TetGen output is a pair of files and 3D only; Triangle output needs a conversion for 2D, or use gmsh.
- Exodus blocks and gmsh physical groups become subdomains: name them, you will want them in chapter 21.
- Keep the mesher's input next to the mesh in the example folder, and make sure it is the input that made the mesh.

# SUMMARY

1. IBFE takes a mesh of the program's dimension, of one of libMesh's element types, with positive Jacobians, placed and scaled in the simulation's units, from a file whose extension picks the reader, or built in the program with libMesh's generators.
2. gmsh makes 2D and 3D meshes from a script or by mouse; MSH 2.2 or 4.1 ASCII, never binary; a counter-clockwise loop and a physical group are the two rules; physical names become subdomain and side-set names.
3. Triangle and TetGen mesh a `.poly` file of nodes, segments or facets, holes and regions; TetGen's `.node`/`.ele` pair reads directly in 3D.
4. CAD goes through Coreform Cubit (Trelis's successor) or gmsh to ExodusII or MSH; Exodus blocks become subdomains.
5. The element-size rules of ten years agree once the elements (solid resolution) and the quadrature points (`IB_point_density`, the coupling) are separated: elements of $\mathrm{DX}$ to $2\,\mathrm{DX}$, points $2$ per cell, more points before smaller elements.
6. Every command here was run today on the course's own files; the counts are on slide 41, the errors on slides 17 and 42.

Chapter 19 writes the stress functions; chapter 21 gives parts of a mesh different materials through the subdomain ids made here; chapter 22 moves a mesh into place with a coordinate mapping.

# RECORDINGS AND LINKS (LISTED, NOT INCORPORATED)

- The lab's July 2022 Coreform Cubit tutorial recording (private Drive; listed in chapter 0.1).
- The 2016 gmsh read-me's links: the Fluidity project's gmsh tutorial (https://github.com/FluidityProject/fluidity/wiki/Gmsh-tutorial) and two YouTube gmsh tutorials ("part 1" `xL2LmDsDLYw`, "part 3" `laKHzos3EbU`); gmsh's own documentation at https://gmsh.info/doc/texinfo/gmsh.html.
- csimsoft's Trelis tutorials and videos (`csimsoft.com/tutorialsTrelis`, dead) and the *Getting Started with Trelis* PDF (2014): see https://coreform.com for their successors.
- Triangle: https://www.cs.cmu.edu/~quake/triangle.html; TetGen: https://wias-berlin.de/software/tetgen/.

# REFERENCES AND CREDITS


- C. Geuzaine and J.-F. Remacle, *Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities*, Int. J. Numer. Meth. Engng 79 (2009) 1309–1331.
- J. R. Shewchuk, *Triangle: engineering a 2D quality mesh generator and Delaunay triangulator*, in *Applied Computational Geometry*, LNCS 1148 (1996) 203–222. H. Si, *TetGen, a Delaunay-based quality tetrahedral mesh generator*, ACM Trans. Math. Softw. 41 (2015) 11.
- libMesh 1.7.8: `include/libmesh/mesh_generation.h`, `src/mesh/gmsh_io.C`, `tetgen_io.C`, `exodusII_io.C`, `namebased_io.C`; https://libmesh.github.io. IBAMR 0.19.0: `src/IB/FEMechanicsBase.cpp` (the Jacobian check), `examples/IBFE/explicit/ex0`–`ex11`.
- The examples of this series: IBFE/05 gmsh triangle (`triangle.geo`, `triangle.msh`, `fix_orientation.py`), IBFE/10 (`plate2.msh`), IBFE/08 (`beam.e`, `circle.e`), IBFE/01 `extras/` (`Beam_IBFE_Mesh2D.m`, `.poly`); `wsl/final/ch18_meshtest.sh`, `ch18_render.sh`.
- Sources: Alex Hoover, *2-IBFE Tutorial: Creating your mesh* (2014–2017); *2.1 - IBFE Tutorial: Make your own mesh and move it!* (2017; MATH 769 2018; lab 2020–2024); Julian Wag, *Using gmsh to make 2D meshes from scratch* (2016; screenshots reproduced with credit); Nick Battista, *How To Make Simple Geometries in Gmsh* (24 September 2014; cited); the meshing, gmsh and Trelis read-mes (2015–2017) and the 2016 Trelis notes; the `cube`, `cone`, `sphere` and `tube .geo` samples of the meshing folder; *5-IBFE Lecture: Plans for June* (2020).